College of Computer and Information Sciences

Department of Information Technology

CY443

Spring 2025

Sr #	Student Name
01	Razan Alshahrani
02	Team member 1
03	Team member 2
04	Team member 3
05	Team member 4

Template for the Project Report

Phishing detection (URLs)

1. Abstract

The phishing attack still remains one of the major social engineering cybersecurity risks that take advantage of humans' weaknesses to hack and steal critical data. Usually, the classic defense techniques make use of blacklisting widely. However, these methods turn out to be inefficient when dealing with identification of novel zero-day phishing websites. In response to that issue, the present paper proposes a new intelligent model for detecting phishing websites employing both Machine Learning (ML) and Deep Learning (DL) techniques.

Firstly, a large amount of data consisting of structural and behavioral features of URLs was cleaned, normalized, and prepared. Then, the classic ML algorithms were built via developing the Decision Tree model and optimizing its predictions using the Random Forest and eXtreme Gradient Boosting (XGBoost) techniques. Besides, the DL pipeline was constructed from the basic ANN model up to advanced DNN model equipped with several optimization layers.
In particular, according to the experiment results, the model based on XGBoost has reached the highest accuracy score equal to 97.18%. At the same time, the proposed DNN model has shown its high effectiveness with the accuracy rate of 95.51% with balanced precision (95.54%), recall (95.51%), and F1-score (95.52%).

2. Introduction and problem statement(s)

2.1 Introduction & Background

With the onset of the digital age, dependency on internet services by both companies and users has grown immensely, coupled with a rise in cyber security threats against such services. One such cyber threat which has proved to be an extremely potent attack vector is phishing, whereby attackers deceive users into divulging sensitive personal data or information via mimicking authentic sources (Mohammad et al., 2015). In the past, cyber security tools used signature-based techniques for analysis, and were dependent on centralized domain blacklisting of malicious websites for protection. Modern day attackers have found ways to evade such conventional security techniques by implementing automated Domain Generation Algorithms (DGA) and fast flux hosting. There has thus arisen an increasing necessity to shift from passive security measures to a more proactive approach, in the form of phishing detection systems that analyze malicious URL properties. With the advent of Artificial Intelligence (AI), particularly machine learning and deep learning algorithms, it has been possible to detect phishing attempts via URL analysis and classification (Aljofey et al., 2022).

2.2 Problem Definition and Chosen Project

For this research project, we focused on phishing detection using URL based features. The main problem addressed in this work is the inability of traditional signature based security tools to detect zero day or rapidly evolving phishing websites. The objective is to build an automated classification system capable of analyzing structural, lexical, and security related anomalies within URLs to accurately determine whether a website is legitimate or phishing before users interact with the malicious page (Yang et al., 2021).

2.3 Dataset Description, Inputs, and Outputs

The project utilizes the “Phishing Detection Cleaned Dataset,” which contains multiple records representing various technical characteristics of websites and domains. The dataset serves as the

foundation for training and evaluating the proposed machine learning and deep learning models.

Inputs: The input features are extracted directly from URLs and include several structural and lexical characteristics commonly associated with phishing websites. These features include HTTPS usage, SSL related anomalies, URL length, domain length, prefix or suffix hyphens, suspicious keywords, embedded IP addresses, and other domain related indicators commonly used in phishing detection research (Christou et al., 2020).
Outputs: The output variable is a binary classification target where:
0 represents a legitimate website
1 represents a phishing website

2.4 Analysis Plan
To systematically evaluate our intelligent security solution, the dataset is first subjected to data preprocessing stages, including feature isolation, missing value verification, and scaling via the StandardScaler normalization technique to eliminate variance bias. The preprocessed data is then partitioned into independent training and testing matrices. The execution and comparative process follows a structured, dual phase methodology.
1. Traditional Machine Learning Framework: We first establish an initial performance benchmark by implementing a standard Decision Tree Classifier. To improve classification performance and reduce overfitting, ensemble learning techniques are then applied through the use of a Random Forest Classifier consisting of 100 randomized trees (n_estimators = 100). Finally, an eXtreme Gradient Boosting (XGBoost) model is implemented to minimize residual learning errors using gradient boosting techniques and regularization parameters such as max_depth = 6 and learning_rate= 0.1 (Yang et al., 2021).

1. Advanced Deep Learning Pipeline: In parallel, a deep learning framework is designed to capture complex and nonlinear phishing related patterns.A basic Artificial Neural Network (ANN) is initially configured as a baseline deep learning model. To further enhance performance, an improved Deep Neural Network (DNN) architecture is developed using multilayer dense structures consisting of 32, 16, and 8 neurons with ReLU activation functions and a final Sigmoid output layer for binary classification. To reduce overfitting during the 100-epoch training process, Dropout layers with a rate of 0.3 and Early Stopping mechanisms are integrated into the Adam optimized training procedure (Aljofey et al., 2022).
1. Performance Metrics Evaluation: The analytical comparison across all implemented machine learning and deep learning models is evaluated using standard classification metrics including Accuracy, Precision, Recall, and F1 Score. Additional evaluation techniques include Confusion Matrices and training convergence curves to validate model generalization capability and reduce false negative classifications in phishing detection systems (Mohammad et al., 2015).

3. Literature Review

Previous research presented some techniques for phishing website detection and explained the concept of phishing websites. A phishing website usually mimics a trusted site in order to acquire sensitive information such as user IDs, passwords, and financial details. Detecting phishing websites before users interact with them is highlighted as the main problem in previous studies (Mohammad et al., 2015). Our work is similar in the sense that it aims to identify whether a URL is phishing or legitimate, except that

instead of reviewing different techniques, we apply and test machine learning and deep learning models on URL features.

A systematic review of phishing website detection techniques presented different approaches, such as blacklist-based detection, heuristic methods, and ML- and DL-based techniques. Traditional blacklist methods are limited since they cannot detect newly created phishing websites (Safi & Singh, 2023). This is relevant to our project because our work also focuses on learning-based detection instead of relying only on traditional methods. Therefore, our work tests various learning-based models to get the best performance for detecting phishing URLs.
Previous work on phishing URL detection using ML-based techniques showed that URLs themselves could represent useful features for detecting suspicious links (Sahingoz et al., 2019). This is relevant in our case as our dataset depends on structured URL features and suggests that ML models can detect phishing patterns from URLs. We first use a Decision Tree as a baseline model, then evaluate improved ML models such as Random Forest and XGBoost to assess performance differences.

The use of ML techniques for malicious URL detection was investigated in another study. The authors of this paper mentioned that preparing the data under the same conditions is crucial. A correct comparison between models can be achieved when the same dataset is evaluated using the same metrics (Reyes-Dorta et al., 2024).This is also applied in our project, where Decision Tree, Random Forest, XGBoost, and DNN are compared.

A deep learning model was proposed to detect phishing web pages using both raw URLs and HTML content. The deep learning model attempted to solve the problem of manual feature extraction by learning the features from the data (Opara et al., 2024). This is related to our project because we also use a deep learning model to detect phishing websites. However, we use structured URL features instead of raw URLs and HTML content. This also suggests a possible future direction for our work, such as using LSTM or Transformer models for raw URL detection.

By observing these studies, we can conclude that the problem of phishing URL detection can be addressed using ML and DL, where URLs represent relevant features. However, a fair comparison of those models is not easy since most of these studies use different datasets, models, and metrics. In our work, we compare a simple Decision Tree model with two improved ML-based models such as Random Forest and XGBoost along with a DNN on the same dataset and the same evaluation metrics, enabling a better comparison of these learning-based models for phishing URL detection.

4. Methods/ Model

4.1 Data Preprocessing

The phishing dataset was cleaned and prepared before model development. The original dataset had 10,000 samples and 50 features. Several features were removed during preprocessing because they were not relevant to the classification task; these features included “ID,” several rule-based features, and “HttpsInHostname.” Following feature selection, duplicate records were removed, resulting in a final dataset of 9,581 samples.

The cleaned dataset had no missing values and no duplicate records. The target classes remained approximately balanced, with 4,996 samples in class 0 and 4,585 samples in class 1; therefore, resampling was not required. Outliers in URL-based features were retained because they reflect valid URL characteristics that may contribute to phishing detection. Feature scaling was left to the modeling stage rather than being applied directly to the dataset file.

4.2 Implemented Models

We implemented four models in total, starting from simple ones and progressing to a more complex one. Firstly, we used a Decision Tree Classifier as a benchmark. A decision tree learns rules in form of if-then statements for the feature values and is thus interpretable and fast. At the same time, it is prone to overfitting when a deeper tree is formed. We used default hyperparameters (random_state=42) with no pruning to get the best performance from it. As a next step, we used a Random Forest consisting of 100 decision trees. In this case, every tree is trained on a random subset of data and features, and a prediction is made based on the majority vote. Compared to a single decision tree, this model overfits less and usually performs much better. We used n_estimators=100 with a fixed random state for reproducibility. Then we implemented XGBoost, an extreme gradient boosting algorithm. It is very similar to Random Forest, except for the fact that new trees are built based on previous mistakes made and thus they learn sequentialy. Moreover, XGBoost usually performs better on tabular data. We chose n_estimators=100, learning_rate=0.1, max_depth=6. Finally, we trained a deep learning neural network the TensorFlow and Keras libraries were used. The neural network has the following structure: input layer, receiving 29 input features; dense layer with 32 neurons with the use of ReLU activation function; dropout layer with rate=0.3 in order to prevent overfitting; dense layer with 16 neurons with the ReLU activation function; dense layer with 8 neurons with the ReLU activation function; output layer with one neuron with the use of sigmoid activation function, which outputs a probability from 0 to 1. The Adam optimization method and the binary cross-entropy were used for compilation of the model. The model was trained for 100 epochs with a batch size of 16. Early stopping technique was used as a means of preventing overfitting in order to stop training when the validation loss no longer improves for 5 consecutive epochs. In addition, the best model weights were saved and loaded into the final model. Twenty percent of the training dataset was selected as the validation set during the training process.

5. Result

We tested all four models using the same held-out test set of 1,917 samples. In addition to accuracy, we provide the results in terms of precision, recall, and F1-score as they give a more comprehensive picture. With respect to the baseline, the Decision Tree model reached 94.94% accuracy. Precision, recall, and F1-score were 0.95 for both classes, which is quite good, though not perfect. This model is prone to memorizing the training set, so it only gives us a baseline score. The Random Forest surpassed the baseline with 96.87% accuracy. Precision, recall, and F1-score increased to 0.97 for both classes due to averaging multiple different trees. XGBoost gave the highest performance among all the models with 97.18% accuracy. For the remaining measures, this model had an identical value of 0.97, too. The bar chart comparing the three machine learning models proves that there is a noticeable performance increase with each method, starting from 94.94% for Decision Tree and ending with 97.18% for XGBoost. As for the neural network, training was interrupted after epoch 44 when the early stopping criterion was met. That is, we got very nice convergence before hitting the maximum allowed number of epochs of 100. The neural network obtained an accuracy of 95.51% on the test dataset, with a precision score of 95.54%, recall score of 95.51%, and F1 score of 95.52%. From the confusion

matrix results, the model successfully predicted 940 true negative cases from the total of 993 genuine URLs, while 891 phishing URLs were identified as true positives out of the total of 924 phishing URLs. On the other hand, there were 53 false positives (genuine URLs marked as phishing) and 33 false negatives (phishing URLs marked as genuine URLs) on the test dataset. False negatives pose more risk in actual application since any phishing URL may be left undetected. Both the training and validation accuracies increase gradually and intersect around 95%, indicating no overfitting of the model.

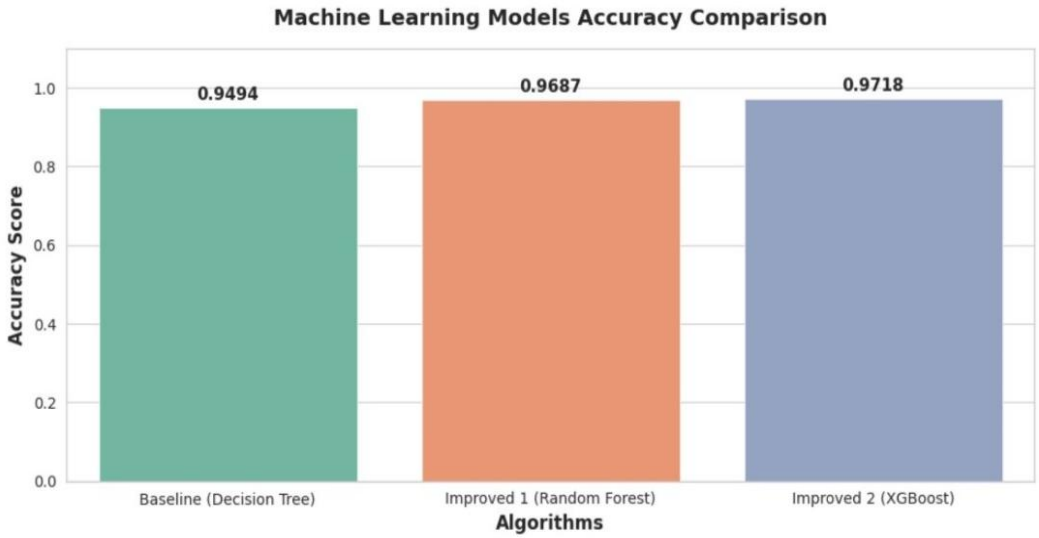


Figure 1 ML model accuracy comparison

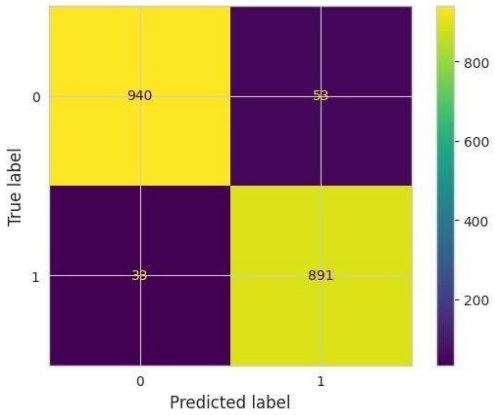


Figure 2 Confusion Matrix of the Improved DNN Model

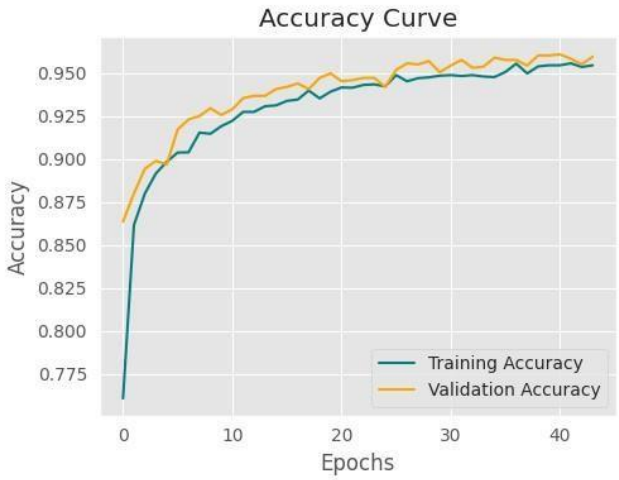


Figure 3 Training and Validation Accuracy Curve of the DNN Model

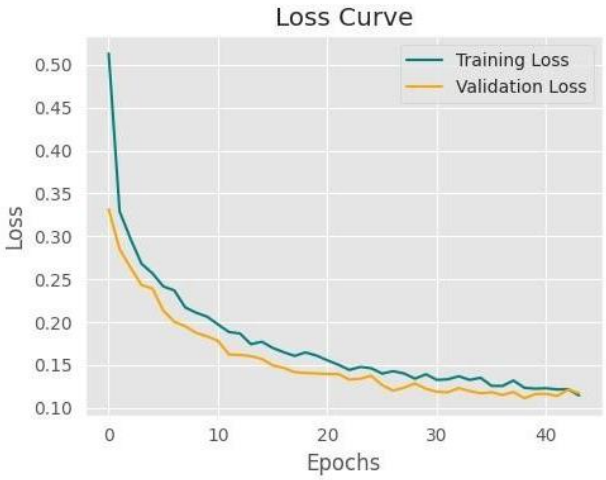


Figure 4 Training and Validation Loss Curve of the DNN Model

Performance of all models:

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree (baseline)	94.94%	0.95	0.95	0.95
Random Forest	96.87%	0.97	0.97	0.97
XGBoots	97.18%	0.97	0.97	0.97
DNN (deep learning)	95.18%	0.955	0.955	0.955

6. Discussion and Analysis

It is evident from the results that gradient boosting algorithms are the best methods to work with the given dataset. Indeed, although the Decision Tree method is interpretable and computationally efficient, it is only based on one model, and hence, it can overfit or ignore some patterns in the data. Random Forest solves this problem by generating many decision trees and taking the average output. Hence, the algorithm goes from 94.94% to 96.87%. However, XGBoost surpasses it by allowing new trees to learn from the errors made by previous trees. It is why the XGBoost algorithm reaches 97.18%. It is interesting to note that the deep learning algorithm works reasonably well with 95.51%, which is still slightly better than the neural network method but worse than any of the ensemble methods. The result comes as no surprise since neural networks are particularly useful with unstructured inputs such as pictures, texts, and sounds, whereas gradient boosting is superior to structured data. The findings are consistent with the existing pattern in this domain. One of the main takeaways from our confusion matrix is the fact that the neural network produced more false positive instances (53) compared to false negatives (33). When applied in phishing, a false negative error is much more critical because it entails that a truly malicious URL gets categorized as harmless. As a result, in an operational setting, we might have to fine-tune our algorithm to minimize the number of false negatives despite having some additional false positives. There are several disadvantages associated with our approach. Firstly, our features are all handcrafted based on the knowledge of common phishing indicators. A well-informed attacker will craft a phishing URL that does not meet any of the criteria and thus avoid being flagged by our model. Secondly, our dataset does not include recent trends in phishing, such as HTTPS-based

phishing websites, which are currently accounting for most of the attacks. Lastly, the architecture used in this study is quite elementary and a more advanced neural network using character embeddings or attention mechanisms might narrow the gap with XGBoost. However, despite the above shortcomings, this project provides proof that even a relatively straightforward application of ML, provided there is enough feature engineering, can yield impressive results for such applications as phishing site detection. Future work may look into identifying the features important for prediction and try out deep learning approaches that would utilize raw URL data directly.

7. References

Aljofey, A., Jiang, Q., Rasool, A., Chen, H., Liu, W., Qu, Q., & Wang, Y. (2022). An effective detection approach for phishing websites using URL and HTML features. Scientific Reports, 12(1), 8842. https://doi.org/10.1038/s41598-022-10841-5

Christou, O., Pitropakis, N., Papadopoulos, P., McKeown, S., & Buchanan, W. J. (2020). Phishing URL detection through top level domain analysis: A descriptive approach. arXiv. https://arxiv.org/abs/2005.06599

Mohammad, R. M., Thabtah, F., & McCluskey, L. (2015). Tutorial and critical analysis of phishing websites methods. Computer Science Review, 17, 1–24. https://doi.org/10.1016/j.cosrev.2015.04.001

Yang, R., Zheng, K., Wu, B., Wu, C., & Wang, X. (2021). Phishing website detection based on deep convolutional neural network and random forest ensemble learning. Sensors, 21(24), 8281. https://doi.org/10.3390/s21248281

Opara, C., Wei, B., & Chen, Y. (2024). Look before you leap: Detecting phishing web pages by exploiting raw URL and HTML characteristics. Expert Systems with Applications, 236, 121183. https://doi.org/10.1016/j.eswa.2023.121183

Reyes-Dorta, N., Caballero-Gil, P., & Rosa-Remedios, C. B. (2024). Detection of malicious URLs using machine learning. Wireless Networks, 30, 7543–7560. https://doi.org/10.1007/s11276-024-03700-w

Safi, A., & Singh, S. (2023). A systematic literature review on phishing website detection techniques. Journal of King Saud University - Computer and Information Sciences, 35, 590–611. https://doi.org/10.1016/j.jksuci.2023.01.004

Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. Expert Systems with Applications, 117, 345–357. https://doi.org/10.1016/j.eswa.2018.09.029